

# CS 677: Distributed and Operating Systems

## Lab 3: Output Document

Manan Parikh (mananrajeshb@umass.edu), Noah Walls (nwalls@umass.edu)

### Basic Demonstration:

Frontend service accessible by EC2 instance's public IP:

```
← → ↻ Not Secure 100.26.110.115:8080/stocks/GameStart
Pretty-print ☒
{
  "data": {
    "name": "GameStart",
    "price": 15.47,
    "quantity": 100
  }
}
```

Running client locally while microservices are running on EC2:

```
< 142eb8357416 spring25-lab3-spring25-lab3-noahw227-mananparikh-client:latest
Logs Inspect Bind mounts Exec Files Stats
2025-05-03 02:52:03 Started execution
2025-05-03 02:52:03 LookUp:
2025-05-03 02:52:03 /stocks/Meta
2025-05-03 02:52:03 Server's Response: {"data":{"name":"Meta","price":164.2,"quantity":120}}
2025-05-03 02:52:04 Duration to complete lookup request (in milliseconds): 200
2025-05-03 02:52:04 Order:
2025-05-03 02:52:04 {"name": "Meta", "quantity": 1, "type" : "buy"}
2025-05-03 02:52:04 Server's Response: {"data":{"transaction_number":33}}
2025-05-03 02:52:04 Duration to complete order request (in milliseconds): 90
2025-05-03 02:52:04 LookUp:
2025-05-03 02:52:04 /stocks/Walmart
2025-05-03 02:52:04 Server's Response: {"data":{"name":"Walmart","price":134.62,"quantity":91}}
2025-05-03 02:52:04 Duration to complete lookup request (in milliseconds): 66
2025-05-03 02:52:04 LookUp:
2025-05-03 02:52:04 /stocks/RottenFishCo
2025-05-03 02:52:04 Server's Response: {"data":{"name":"RottenFishCo","price":6.23,"quantity":236}}
2025-05-03 02:52:04 Duration to complete lookup request (in milliseconds): 68
2025-05-03 02:52:04 Order:
2025-05-03 02:52:04 {"name": "RottenFishCo", "quantity": 30, "type" : "buy"}
2025-05-03 02:52:04 Server's Response: {"data":{"transaction_number":34}}
2025-05-03 02:52:04 Duration to complete order request (in milliseconds): 79
2025-05-03 02:52:04 LookUp:
2025-05-03 02:52:04 /stocks/Google
2025-05-03 02:52:04 Server's Response: {"data":{"name":"Google","price":212.39,"quantity":380}}
2025-05-03 02:52:04 Duration to complete lookup request (in milliseconds): 90
2025-05-03 02:52:04
```

### Cache Replacement in Action:

Running test case *testMultipleLookUpRequests* sends a sequence of stock lookup requests is as follows:

1-GameStart, 2-Pfizer, 3-RottenFishCo, 4-GameStart (again), 5-Google, 6-Alphabet, 7-Walmart

We've set cache size to 5. Once the cache reaches its max capacity (after the "Alphabet" request), a new request will trigger a replacement based on LRU policy. When the "Walmart" request comes in, the cache is already full, so the least recently used entry, "Pfizer", will be evicted. "GameStart" was accessed twice and is still considered recently used, so it will remain in the cache. The frontend service logs for this scenario are as follows:

```
Received lookup request for GameStart
Cache lookup for GameStart: not found.
Adding GameStart to cache.
Moved GameStart to the head of the cache.

Received lookup request for Pfizer
Cache lookup for Pfizer: not found.
Adding Pfizer to cache.
Moved Pfizer to the head of the cache.

Received lookup request for RottenFishCo
Cache lookup for RottenFishCo: not found.
Adding RottenFishCo to cache.
Moved RottenFishCo to the head of the cache.

Received lookup request for GameStart
Cache lookup for GameStart: {data={"name":"GameStart","price":15.47,"quantity":100}}

Received lookup request for Google
Cache lookup for Google: not found.
Adding Google to cache.
Moved Google to the head of the cache.

Received lookup request for Alphabet
Cache lookup for Alphabet: not found.
Adding Alphabet to cache.
Moved Alphabet to the head of the cache.

Received lookup request for Walmart
Cache lookup for Walmart: not found.
Adding Walmart to cache.
Cache capacity exceeded. Removing entry for Pfizer
Moved Walmart to the head of the cache.
fec2-user@ip-172-31-26-203 lab31$
```

### Simulating Leader Crash:

#### During Client Order Requests:

Killing the leader order service replica during client request, restarting after some time:

```
[ec2-user@ip-172-31-26-203 lab3]$ docker stop 29dc
29dc
[ec2-user@ip-172-31-26-203 lab3]$ docker start 29dc
29dc
[ec2-user@ip-172-31-26-203 lab3]$
```

Frontend logs indicate that the frontend detected the leader replica failure and elected a new leader:

```
[ec2-user@ip-172-31-26-203 lab3]$ docker logs d33 | grep -C 8 down
Received lookup request for GameStart
Cache lookup for GameStart: not found.
Adding GameStart to cache.
Cache capacity exceeded. Removing entry for Google
Moved GameStart to the head of the cache.

Received trade request: {"name": "GameStart", "quantity": 20, "type": "buy"}
Error sending request to order service.
Leader is down, electing new leader...
Pinging Order Service 2: http://orderservice2:8082
Error ping Order Service 2
Order Service 2 is not available.
Pinging Order Service 1: http://orderservice1:8082
Order Service 1 is available
New Order Service Leader: http://orderservice1:8082

Received invalidation request from catalog-service and successfully removed the cache entry for stock: GameStart
[ec2-user@ip-172-31-26-203 lab3]$
```

Client logs show that this order {GameStart, 20, buy} was completed successfully with only a small increase in response time (207 ms for this order vs 81ms for the next order):

```
2025-05-03 03:20:45 LookUp:
2025-05-03 03:20:45 /stocks/GameStart
2025-05-03 03:20:45 Server's Response: {"data":{"name":"GameStart","price":15.47,"quantity":98}}
2025-05-03 03:20:45 Duration to complete lookup request (in milliseconds): 68
2025-05-03 03:20:45
2025-05-03 03:20:45 Order:
2025-05-03 03:20:45 {"name": "GameStart", "quantity": 20, "type": "buy"}
2025-05-03 03:20:46 Server's Response: {"data":{"transaction_number":21}}
2025-05-03 03:20:46 Duration to complete order request (in milliseconds): 207
2025-05-03 03:20:46
2025-05-03 03:20:46 LookUp:
2025-05-03 03:20:46 /stocks/Meta
2025-05-03 03:20:46 Server's Response: {"data":{"name":"Meta","price":164.2,"quantity":95}}
2025-05-03 03:20:46 Duration to complete lookup request (in milliseconds): 68
2025-05-03 03:20:46
2025-05-03 03:20:46 Order:
2025-05-03 03:20:46 {"name": "Meta", "quantity": 120, "type": "buy"}
2025-05-03 03:20:46 Server's Response: {"error":{"message":"order request could not be completed.,"code":404}}
2025-05-03 03:20:46 Duration to complete order request (in milliseconds): 81
2025-05-03 03:20:46
```

Order logs (orders.csv files) of all the order services after simulating leader failure indicate data is consistent:

```
[ec2-user@ip-172-31-26-203 lab3]$ docker exec 29dc tail -5 data/orders.csv
167,Pfizer,120,sell
168,Alphabet,2,sell
169,Amazon,250,sell
170,Walmart,230,sell
171,Amazon,1,sell
[ec2-user@ip-172-31-26-203 lab3]$ docker exec e038 tail -5 data/orders.csv
167,Pfizer,120,sell
168,Alphabet,2,sell
169,Amazon,250,sell
170,Walmart,230,sell
171,Amazon,1,sell
[ec2-user@ip-172-31-26-203 lab3]$ docker exec f5ae tail -5 data/orders.csv
167,Pfizer,120,sell
168,Alphabet,2,sell
169,Amazon,250,sell
170,Walmart,230,sell
171,Amazon,1,sell
[ec2-user@ip-172-31-26-203 lab3]$
```

#### During Client Order LookUp Requests:

The Client will still be able to receive response from the new elected leader and continue sending more requests via frontend service. The client will not be able to notice any significant delay.

```

Received lookup request for Apple
Cache lookup for Apple: not found.
Adding Apple to cache.
Moved Apple to the head of the cache.
Order Query request received from client
Order Query request received from client
Error reading response in JsonResponse.java: connect timed out
Error sending request to order service.
Leader is down, electing new leader...
Pinging Order Service 2: http://orderservice2:8082
Error ping Order Service 2
Order Service 2 is not available.
Pinging Order Service 1: http://orderservice1:8082
Order Service 1 is available
New Order Service Leader: http://orderservice1:8082
Order Query request received from client
Order Query request received from client
Order Query request received from client

```

Order lookup requests

Leader is down

### Simulating Follower Failure:

Initially the leader is orderservice2. When we kill one of the follower replicas (orderservice1 in this example), and bring it back online after some time, all orders.csv files should have the same content.

```

[ec2-user@ip-172-31-26-203 lab3]$ docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED        STATUS        PO
d330d2fe2dbc   lab3-frontendservice "java -jar frontend-..." 47 minutes ago Up 16 minutes 0.
1
29dc8d947d25   lab3-orderservice2   "java -jar order-ser..." 47 minutes ago Up 16 minutes 0.
e03853c24c96   lab3-orderservice1   "java -jar order-ser..." 47 minutes ago Up 16 minutes 0.
9045946dbbcb   lab3-catalogservice  "java -jar catalog-s..." 47 minutes ago Up 16 minutes 0.
f5ae288abe45   lab3-orderservice0   "java -jar order-ser..." 47 minutes ago Up 16 minutes 0.
[ec2-user@ip-172-31-26-203 lab3]$ docker stop e0385
e0385
[ec2-user@ip-172-31-26-203 lab3]$ docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED        STATUS        PO
d330d2fe2dbc   lab3-frontendservice "java -jar frontend-..." 47 minutes ago Up 16 minutes 0.
1
29dc8d947d25   lab3-orderservice2   "java -jar order-ser..." 47 minutes ago Up 16 minutes 0.
9045946dbbcb   lab3-catalogservice  "java -jar catalog-s..." 47 minutes ago Up 16 minutes 0.
f5ae288abe45   lab3-orderservice0   "java -jar order-ser..." 47 minutes ago Up 16 minutes 0.
[ec2-user@ip-172-31-26-203 lab3]$

```

By running `docker logs 29dc` (29dc is the process ID of orderservice2), we get logs of orderservice2, which is clearly able to identify that orderservice1 went down during the order {Amazon, 120, sell}

```

[ec2-user@ip-172-31-26-203 ~]$ docker logs 29dc | grep -C 8 Error
UpdateFollowers: Update sent to follower 1 successfully.

Received trade request: {"name": "RottenFishCo", "quantity": 25, "type": "buy"}
UpdateFollowers: Update sent to follower 0 successfully.
UpdateFollowers: Update sent to follower 1 successfully.

Received trade request: {"name": "Amazon", "quantity": 120, "type": "sell"}
UpdateFollowers: Update sent to follower 0 successfully.
UpdateFollowers: Error sending update to follower 1

```

Bringing orderservice1 back online again, orderservice2 receives its request for missing orders (orders after transaction number 100 in this case), and returns those orders.

```

Received trade request: {"name": "Pfizer", "quantity": 1, "type": "sell"}
UpdateFollowers: Update sent to follower 0 successfully.
UpdateFollowers: Error sending update to follower 1

Received trade request: {"name": "Apple", "quantity": 230, "type": "buy"}
UpdateFollowers: Update sent to follower 0 successfully.
UpdateFollowers: Error sending update to follower 1

Received trade request: {"name": "Amazon", "quantity": 1, "type": "sell"}
UpdateFollowers: Update sent to follower 0 successfully.
InformHandler: Order found for transaction number 100
UpdateFollowers: Update sent to follower 1 successfully.

Received trade request: {"name": "Google", "quantity": 120, "type": "sell"}
UpdateFollowers: Update sent to follower 0 successfully.
UpdateFollowers: Update sent to follower 1 successfully.

Received trade request: {"name": "Airbnb", "quantity": 20, "type": "buy"}
UpdateFollowers: Update sent to follower 0 successfully.
UpdateFollowers: Update sent to follower 1 successfully.

```

The client is not really affected as the order {Amazon, 120, sell} only takes 82 ms to complete. This around the normal response time even for an order that was not interrupted by a crash.

```

LookUp:
/stocks/Amazon
Server's Response: {"data":{"name":"Amazon","price":172.61,"quantity":9640}}
Duration to complete lookup request (in milliseconds): 66

Order:
{"name": "Amazon", "quantity": 120, "type": "sell"}
Server's Response: {"data":{"transaction_number":73}}
Duration to complete order request (in milliseconds): 82

```

Orders.csv file of all the order-services after simulating follower failure

```
[ec2-user@ip-172-31-26-203 lab3]$ docker exec lab3-orderservice1-1 tail -5 data/orders.csv
195,Amazon,120,sell
196,Amazon,120,sell
197,Amazon,120,sell
198,Amazon,120,sell
199,Amazon,120,sell
[ec2-user@ip-172-31-26-203 lab3]$ docker exec lab3-orderservice2-1 tail -5 data/orders.csv
195,Amazon,120,sell
196,Amazon,120,sell
197,Amazon,120,sell
198,Amazon,120,sell
199,Amazon,120,sell
[ec2-user@ip-172-31-26-203 lab3]$ docker exec lab3-orderservice0-1 tail -5 data/orders.csv
195,Amazon,120,sell
196,Amazon,120,sell
197,Amazon,120,sell
198,Amazon,120,sell
199,Amazon,120,sell
```

### **Evaluation Fault-Tolerance Questions:**

**Can the clients notice the failures? (either during order requests or the final order checking phase) or are they transparent to the clients?**

The client will experience some delay due to replica crashes during most order related requests but these delays are generally too small to notice (usually around 30 ms, maximum of 200 ms).

- **Leader Crash During Order Request** - If the leader suddenly goes down during a client order request, the client will notice a minor delay (around 50-200 milliseconds) to get the response back.
- **Leader Crash During Order Lookup Request** - If the leader crashes during an order lookup request, the client will notice a similar delay to the order request case as the same method of leader re-election will be used. However there will be slightly less delay (about 30ms) as the new leader will not attempt to push an order update to the remaining followers.
- **Follower Crash During Order Request** - If any of the follower replicas go down while the client is making requests the client won't see any noticeable difference. After receiving an order request from the frontend, the leader will attempt to push updates to the followers before responding to the frontend. Timeouts are implemented to force the leader to give up on the update attempt after 30 ms if the follower is unresponsive. This results in a 30 ms delay, which will not be noticeable to the client.
- **Follower Crash During Order Lookup Request** - Followers are not contacted by the leader replica for an order lookup request. There is no impact on a client making these requests if a follower crashes.

**Do all the order service replicas end up with the same database file?**

As demonstrated in the above screenshots, in both scenarios (follower and leader crash), all replicas end up with the same data in their logs. There may be edge cases where the data becomes inconsistent but they are outside of the scope of this lab.